



# **EC200D&ECx00G&EC600U&EG800G Series QuecOpen USBnet Call API Reference Manual**

**LTE Standard Module Series**

Version: 1.0

Date: 2023-08-17

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

**Quectel Wireless Solutions Co., Ltd.**

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: [info@quectel.com](mailto:info@quectel.com)

**Or our local offices. For more information, please visit:**

<http://www.quectel.com/support/sales.htm>.

**For technical support, or to report documentation errors, please visit:**

<http://www.quectel.com/support/technical.htm>.

Or email us at: [support@quectel.com](mailto:support@quectel.com).

## Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

## Use and Disclosure Restrictions

### License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

### Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

## Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

## Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

## Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

## Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

**Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.**

# About the Document

## Revision History

| Version | Date       | Author | Description              |
|---------|------------|--------|--------------------------|
| -       | 2023-08-01 | Joe TU | Creation of the document |
| 1.0     | 2023-08-17 | Joe TU | First official release   |

## Contents

|                                       |           |
|---------------------------------------|-----------|
| About the Document.....               | 3         |
| Contents .....                        | 4         |
| Table Index.....                      | 5         |
| Figure Index .....                    | 6         |
| <b>1 Introduction .....</b>           | <b>7</b>  |
| 1.1. Applicable Modules .....         | 7         |
| <b>2 USBnet Calling Process .....</b> | <b>8</b>  |
| <b>3 USBnet Call API.....</b>         | <b>10</b> |
| 3.1. Header File .....                | 10        |
| 3.2. API Overview .....               | 10        |
| 3.3. API Description.....             | 11        |
| 3.3.1. ql_usbnet_set_type.....        | 11        |
| 3.3.1.1. ql_usbnet_type_e .....       | 11        |
| 3.3.1.2. ql_usbnet_errcode_e.....     | 12        |
| 3.3.2. ql_usbnet_get_type.....        | 13        |
| 3.3.3. ql_usbnet_start.....           | 13        |
| 3.3.3.1. ql_data_call_conf_s.....     | 14        |
| 3.3.4. ql_usbnet_stop .....           | 14        |
| 3.3.5. ql_usbnet_get_status .....     | 15        |
| 3.3.5.1. ql_usbnet_state_e .....      | 15        |
| 3.3.6. ql_usbnet_register_cb.....     | 16        |
| 3.3.6.1. ql_usbnet_callback .....     | 16        |
| <b>4 Example .....</b>                | <b>18</b> |
| 4.1. USBnet Calling Demo .....        | 18        |
| 4.2. Demo Auto-Start.....             | 18        |
| <b>5 Appendix References .....</b>    | <b>19</b> |

**Table Index**

Table 1: Applicable Modules ..... 7

Table 2: API Overview ..... 10

Table 3: Related Documents ..... 19

Table 4: Terms and Abbreviations ..... 19

**Figure Index**

Figure 1: USBnet Calling Process ..... 8

Figure 2: Demo Auto-Start ..... 18

# 1 Introduction

Quectel EC200D, EC600U, EG800G and ECx00G family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **documents [1]** and **[2]**.

This document explains the USBnet call functionality, including API calling process, USBnet API and the example of Quectel EC200D, EC600U, EG800G and ECx00G family modules in QuecOpen® solution.

## 1.1. Applicable Modules

**Table 1: Applicable Modules**

| Module Family | Module        |
|---------------|---------------|
| -             | EC200D Series |
| -             | EC600U Series |
| ECx00G        | EC200G-CN     |
|               | EC600G Series |
|               | EC800G-CN     |
| -             | EG800G Series |

### NOTE

For the EC200D series module, the file paths mentioned in this document do not contain *components\*.



## 2 USBnet Calling Process

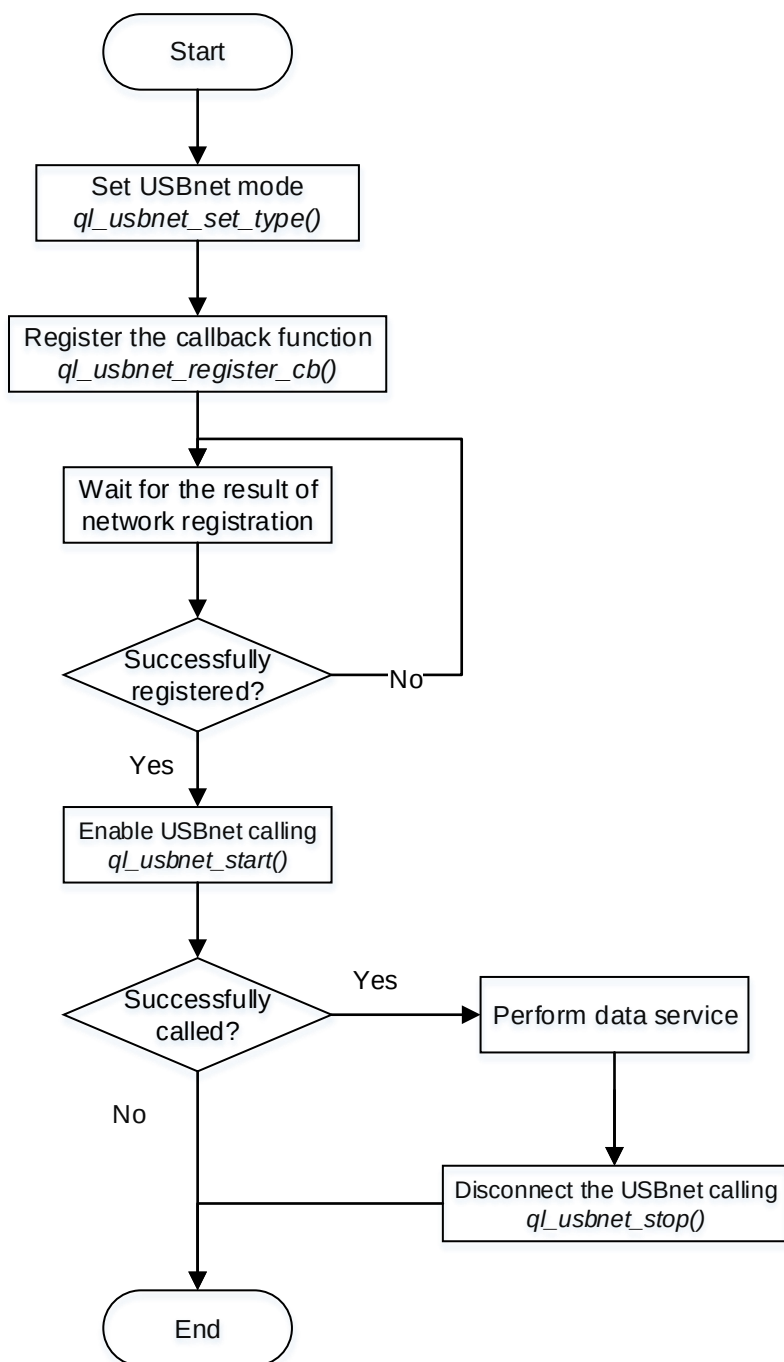


Figure 1: USBnet Calling Process

**NOTE**

Network registration is finished automatically without code intervention. You just need to call `ql_network_register_wait()` to wait for the completion of network registration. You can also check if the network registration is successful by calling `ql_nw_register_cb()` to register the network event callback function. For details about `ql_nw_register_cb()` and `ql_network_register_wait()`, see **documents [3]** and **[4]** respectively.

# 3 USBnet Call API

## 3.1. Header File

*ql\_api\_usbnet.h*, the header file of USBnet call API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, the header files mentioned in this document are all located in this directory.

## 3.2. API Overview

Table 2: API Overview

| Function                       | Description                                  |
|--------------------------------|----------------------------------------------|
| <i>ql_usbnet_set_type()</i>    | Sets USBnet mode.                            |
| <i>ql_usbnet_get_type()</i>    | Gets the current USBnet mode.                |
| <i>ql_usbnet_start()</i>       | Starts a USBnet call.                        |
| <i>ql_usbnet_stop()</i>        | Stops a USBnet call.                         |
| <i>ql_usbnet_get_status()</i>  | Gets the connection status of a USBnet call. |
| <i>ql_usbnet_register_cb()</i> | Registers the USBnet call callback function. |

## 3.3. API Description

### 3.3.1. ql\_usbnet\_set\_type

This function sets USBnet mode and, currently, only RNDIS and ECM are supported. Once the function is called, it takes effect after the module is rebooted.

- **Prototype**

```
ql_usbnet_errcode_e ql_usbnet_set_type(ql_usbnet_type_e usbnet_type)
```

- **Parameter**

*usbnet\_type*:

[In] USBnet mode to be set. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.2** for details.

#### 3.3.1.1. ql\_usbnet\_type\_e

The enumeration of USBnet modes is defined as below:

```
typedef enum
{
    QL_USBNET_NONE = 0,
    QL_USBNET_ECM,
    QL_USBNET_MBIM,
    QL_USBNET_RNDIS,
    QL_USBNET_MAX
}ql_usbnet_type_e;
```

- **Member**

| Member                 | Description                |
|------------------------|----------------------------|
| <i>QL_USBNET_ECM</i>   | ECM mode.                  |
| <i>QL_USBNET_MBIM</i>  | MBIM mode (not supported). |
| <i>QL_USBNET_RNDIS</i> | RNDIS mode.                |

### 3.3.1.2. ql\_usbnet\_errcode\_e

The result codes of USBnet call indicate if the function is executed successfully. The enumeration of result codes is defined as below:

```
typedef enum
{
    QL_USBNET_SUCCESS                = 0,
    QL_USBNET_EXECUTE_ERR            = 1 | QL_USBNET_ERRCODE_BASE,
    QL_USBNET_MEM_ADDR_NULL_ERR,
    QL_USBNET_INVALID_PARAM_ERR,
    QL_USBNET_USB_NOT_CONNECT_ERR,
    QL_USBNET_PDP_ACTIVE_ERR        = 5 | QL_USBNET_ERRCODE_BASE,
    QL_USBNET_REPEAT_CONNECT_ERR,
    QL_USBNET_REPEAT_DISCONNECT_ERR,
}ql_usbnet_errcode_e;
```

#### ● Member

| Member                                 | Description                |
|----------------------------------------|----------------------------|
| <i>QL_USBNET_SUCCESS</i>               | Successful execution.      |
| <i>QL_USBNET_EXECUTE_ERR</i>           | Failed execution.          |
| <i>QL_USBNET_MEM_ADDR_NULL_ERR</i>     | Parameter address is NULL. |
| <i>QL_USBNET_INVALID_PARAM_ERR</i>     | Invalid parameter.         |
| <i>QL_USBNET_USB_NOT_CONNECT_ERR</i>   | Unconnected USB.           |
| <i>QL_USBNET_PDP_ACTIVE_ERR</i>        | Failed to activate PDP.    |
| <i>QL_USBNET_REPEAT_CONNECT_ERR</i>    | Connect repeatedly.        |
| <i>QL_USBNET_REPEAT_DISCONNECT_ERR</i> | Disconnect repeatedly.     |

### 3.3.2. ql\_usbnet\_get\_type

This function gets the current USBnet mode.

- **Prototype**

```
ql_usbnet_errcode_e ql_usbnet_get_type(ql_usbnet_type_e *usbnet_type)
```

- **Parameter**

*usbnet\_type*:

[Out] USBnet mode. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.2** for details.

### 3.3.3. ql\_usbnet\_start

This function starts a USBnet call. *QL\_USBNET\_SUCCESS* returned after function execution does not indicate a successful establishment of a call, but only a successful function execution. The registered callback function notifies the client Application if the call has been established successfully with *QUEC\_USBNET\_START\_RSP\_IND*. See **Chapter 3.3.6.1** for details.

- **Prototype**

```
ql_usbnet_errcode_e ql_usbnet_start(uint8_t nSim, int profile_idx, ql_data_call_conf_s *config)
```

- **Parameter**

*nSim*:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, the parameter can be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*profile\_idx*:

[In] PDP context ID. Range: 1–7.

*config*:

[In] Data call configuration. See **Chapter 3.3.3.1** for details. If the parameter is NULL, use default configuration.

- Return Value

See **Chapter 3.3.1.2** for details.

### 3.3.3.1. ql\_data\_call\_conf\_s

The structure of data call configuration is defined as below:

```
typedef struct
{
    int ip_version;
    char apn_name[APN_LEN_MAX];
    char username[USERNAME_LEN_MAX];
    char password[PASSWORD_LEN_MAX];
    int auth_type;
}ql_data_call_conf_s;
```

- Parameter

| Type | Parameter         | Description                                                                                                                     |
|------|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| int  | <i>ip_version</i> | IP type.<br><u>1</u> IPv4<br>2 IPv6<br>3 IPv4v6                                                                                 |
| char | <i>apn_name</i>   | Access Point Name. Maximum length: 64 bytes.                                                                                    |
| char | <i>username</i>   | Username. Maximum length: 64 bytes.                                                                                             |
| char | <i>password</i>   | Password. Maximum length: 64 bytes.                                                                                             |
| int  | <i>auth_type</i>  | Authentication type.<br><u>0</u> No authentication protocols<br>1 PAP authentication protocol<br>2 CHAP authentication protocol |

### 3.3.4. ql\_usbnet\_stop

This function stops a USBnet call. After the call is stopped, the USBnet switches back to initial status.

- Prototype

```
ql_usbnet_errcode_e ql_usbnet_stop(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 3.3.1.2** for details.

### 3.3.5. ql\_usbnet\_get\_status

This function gets the connection status of a USBnet call.

- **Prototype**

```
ql_usbnet_errcode_e ql_usbnet_get_status(ql_usbnet_state_e *status)
```

- **Parameter**

*status*:

[Out] Connection status of a USBnet call. See **Chapter 3.3.5.1** for details.

- **Return Value**

See **Chapter 3.3.1.2** for details.

#### 3.3.5.1. ql\_usbnet\_state\_e

The enumeration of connection status of a USBnet call is defined as below:

```
typedef enum
{
    QL_USBNET_STATE_NONE = 0,
    QL_USBNET_STATE_START,
    QL_USBNET_STATE_CONNECT,
    QL_USBNET_STATE_PORT_DISCONNECT,
    QL_USBNET_STATE_MAX,
}ql_usbnet_state_e;
```

- **Member**

| Member               | Description     |
|----------------------|-----------------|
| QL_USBNET_STATE_NONE | Initial status. |



|                                              |                                |
|----------------------------------------------|--------------------------------|
| <code>QL_USBNET_STATE_START</code>           | The USBnet is connecting.      |
| <code>QL_USBNET_STATE_CONNECT</code>         | Successful connection.         |
| <code>QL_USBNET_STATE_PORT_DISCONNECT</code> | The USB port is not connected. |

### 3.3.6. ql\_usbnet\_register\_cb

This function registers the USBnet call callback function. The USBnet notifies the client Application of the event with the registered callback function.

#### ● Prototype

```
ql_usbnet_errcode_e ql_usbnet_register_cb(ql_usbnet_callback usbnet_cb, void *ctx)
```

#### ● Parameter

*usbnet\_cb*:

[In] Callback function of USBnet call event. See **Chapter 3.3.6.1** for function definition.

*ctx*:

[In] Argument pointer of callback function.

#### ● Return Value

See **Chapter 3.3.1.2** for details.

#### 3.3.6.1. ql\_usbnet\_callback

This function is the callback function of USBnet call event.

#### ● Prototype

```
typedef void (*ql_usbnet_callback)(unsigned int ind_type, ql_usbnet_errcode_e errcode, void *ctx)
```

#### ● Parameter

*ind\_type*:

[In] USBnet call event type.

|                                              |                                                                                    |
|----------------------------------------------|------------------------------------------------------------------------------------|
| <code>QUEC_USBNET_START_RSP_IND</code>       | USBnet call start response event.                                                  |
| <code>QUEC_USBNET_DEACTIVE_IND</code>        | USBnet disconnection event including network signal loss or network disconnection. |
| <code>QUEC_USBNET_PORT_CONNECT_IND</code>    | USB port connection event.                                                         |
| <code>QUEC_USBNET_PORT_DISCONNECT_IND</code> | USB port disconnection event.                                                      |

*errcode:*

[In] Function execution result codes. See **Chapter 3.3.1.2** for details.

*ctx:*

[In] Argument pointer of callback function.

# 4 Example

## 4.1. USBnet Calling Demo

*usbnet\_demo.c*, the example file of USBnet call provided in the SDK of Quectel EC200D, EC600U, EG800G and ECx00G family QuecOpen modules, is in the *components\ql-application\usbnet* directory.

## 4.2. Demo Auto-Start

Uncomment *ql\_usbnet\_app\_init()* in *ql\_init\_demo\_thread()* to start the Demo. The Demo can start automatically at bootup after the compiled firmware is downloaded to the module.

```

468:
469: #ifdef QL_APP_FEATURE_USBNET
470: » //ql_usbnet_app_init();
471: #endif
472:
473: #ifdef QL_APP_FEATURE_SFTP
474: » //ql_sftp_app_init();
475: #endif
476:
477:     ql_rtos_task_sleep_ms(1000); /*Chaos change: set to 1000 for the camera power on*/
478:     ql_rtos_task_delete(NULL);
479: } « end ql_init_demo_thread »
480:

```

Figure 2: Demo Auto-Start

# 5 Appendix References

**Table 3: Related Documents**

| Document Name                                                                                      |
|----------------------------------------------------------------------------------------------------|
| [1] Quectel_ECx00G&EC600U&EG800G_Series_QuecOpen_CSDK_Quick_Start_Guide                            |
| [2] Quectel_EC200D_Series_QuecOpen_CSDK_Quick_Start_Guide                                          |
| [3] Quectel_ECx00G&EC600U&EG800G_Series_QuecOpen_Cellular_Network_Information_API_Reference_Manual |
| [4] Quectel_EC200D&ECx00G&EC600U&EG800G_Series_QuecOpen_Data_Call_API_Reference_Manual             |

**Table 4: Terms and Abbreviations**

| Abbreviation | Description                                   |
|--------------|-----------------------------------------------|
| API          | Application Programming Interface             |
| CHAP         | Challenge Handshake Authentication Protocol   |
| ECM          | Ethernet Networking Control Model             |
| IPv4         | Internet Protocol Version 4                   |
| IPv6         | Internet Protocol Version 6                   |
| MBIM         | Mobile Broadband Interface Model              |
| PAP          | Password Authentication Protocol              |
| PDP          | Packet Data Protocol                          |
| RNDIS        | Remote Network Driver Interface Specification |
| SDK          | Software Development Kit                      |
| IoT          | Internet of Things                            |

USB

Universal Serial Bus

(U)SIM

(Universal) Subscriber Identity Module

---